Documentación Técnica del Cliente POSIX

Módulo de Red, Serialización de Protocolo y Reproducción Multimedia

– Protocolo, Redes y Cliente

02-11-26

1. Introducción y Alcance

El presente documento detalla el diseño, la implementación y la verificación de la capa del cliente y del protocolo de comunicación binario en C/POSIX para el sistema de transmisión y descarga de contenidos de video. El módulo desarrollado bajo la responsabilidad de la **Persona 1** incluye el análisis de la interfaz de línea de comandos (CLI), la gestión del flujo de datos sobre sockets TCP, la serialización/deserialización del protocolo binario de 14 bytes y la integración con el reproductor multimedia `VLC` a través de tuberías IPC.

La Sección 5 documenta además las funciones auxiliares de la capa de servicio (archivos `catalogo.c` y `sesiones.c` de la carpeta `server`). Esas funciones las usa el *servidor*, no el cliente; se incluyen aquí como referencia del equipo.

2. Mapeo de Requerimientos Cumplidos

A continuación se contrasta el código desarrollado frente a los Requerimientos Funcionales (RF) y No Funcionales (RNF) especificados en el proyecto. Los requerimientos marcados como **(Parcial)** se cumplen solo en parte; lo que falta se detalla en la Sección 9.

3. Arquitectura del Protocolo de Red

3.1. Estructura de la Cabecera Binaria (14 Bytes)

Para alinearse con la especificación del protocolo de la Sección 7.2 del proyecto, la cabecera se empaqueta de forma estricta evitando el relleno de la alineación de memoria de C mediante el atributo `__attribute__((packed))`. Todos los campos viajan en orden de red (*Big-Endian*):

```
typedef struct {
    uint16_t version;   // Version del protocolo (1)
    uint16_t tipo;      // Tipo de mensaje (MSG_SALUDO, MSG_ABRIR, etc.)
    uint32_t longitud;  // Longitud de la carga util (Payload)
    uint16_t estado;    // Codigo de estado devuelto por el servidor
    uint32_t id;        // Identificador de la solicitud
} __attribute__((packed)) HeaderProtocolo;
```

3.2. Tipos de Mensaje y Carga Útil

Después de la cabecera viajan exactamente `longitud` bytes de carga útil. El receptor debe leerlos siempre, aunque no los use; de lo contrario, el siguiente mensaje se interpretaría a partir de bytes equivocados.

El mensaje FIN se llama `MSG_FINAL` en el código porque `<sys/socket.h>` ya define una constante `MSG_FIN`; con ese nombre el programa no compila sin avisos.

Requerimiento	Implementación y Justificación Técnica en `cliente.c`
RF-08	**Solicitud por Nombre:** La acción `descargar` envía `MSG_ABRIR` con el nombre del contenido (`-f`) tal como figura en el catálogo.
RF-11	**Consulta de Catálogo:** La acción `listar` envía `MSG_LISTAR` y muestra el nombre y el tamaño de cada contenido recibido en `MSG_CATALOGO`.
RF-24 / RF-64	**Interfaz CLI de Tres Acciones:** Implementación mediante bucle de análisis de argumentos capturando IP (`-h`), puerto (`-p`), acción (`-a`), archivo (`-f`), destino (`-d`) y bandera para no reproducir (`-n`). Los argumentos se validan antes de abrir el socket.
RF-24-bis	**Integración con VLC:** Detección del ejecutable mediante `vlc_disponible()` y lanzamiento con `lanzar_vlc()` (`fork` + `exec`). En el canal en vivo los bytes recibidos se escriben en la entrada de VLC por una tubería (`pipe` + `dup2`); en la descarga, VLC abre el archivo al terminar.
RF-25 / RF-67	**Validación Paramétrica:** Verificación de presencia y formato de variables requeridas. Retorno explícito con código `EXIT_ARGS_ERROR` (1) si faltan parámetros, si el puerto está fuera de rango, si la acción es desconocida o si `descargar` no trae `-f` y `-d`. **(Parcial)**: el destino no escribible se detecta después de conectarse.
RF-26 / RF-66	**Ayuda del Sistema:** Función `mostrar_ayuda()` invocable explícitamente mediante `-help` o ante sintaxis malformada, con un ejemplo por acción. **(Parcial)**: no indica valores por omisión.
RF-29 / RF-68	**Códigos de Salida:** Éxito (`0`), Error de Argumentos (`1`), Error de Red (`2`) y Error de Contenido (`3`); ver Sección 7. Cumple RF-29. **(Parcial para RF-68)**: no hay códigos propios para rechazo por capacidad ni para error interno.
RF-36 / RF-70	**Limpieza de Archivos Parciales:** En la acción `descargar`, si la conexión cae, el servidor aborta o se detiene a mitad de transferencia, el cliente elimina el archivo truncado mediante `remove(destino)`. **(Parcial)**: si el usuario interrumpe con Ctrl+C, el archivo parcial queda en disco.
RF-37	**Verificación de Integridad:** Comprobación exacta entre los bytes leídos en el flujo (`tam_recibido`) y el tamaño total anunciado por el servidor (`be64(tam_be)`) al recibir el mensaje `MSG_FINAL`. Una discrepancia se informa por la salida de error y la descarga se descarta.
RF-69	**Separación de Flujos:** Los mensajes informativos se imprimen con `printf` (salida estándar) y los errores con `fprintf(stderr, ...)`.
RNF-11 / RNF-15	**Recursos y Lectura Exacta:** Cierre de sockets mediante `close(sockfd)`, de archivos mediante `fclose()` y de tuberías mediante `close(fd_vlc)`. Todas las cabeceras se leen y escriben completas con `leer_exacto()` y `escribir_exacto()`.

Cuadro 1: Matriz de trazabilidad entre Requerimientos del Proyecto y el Código C.

Los códigos de estado del campo `estado` (Sección 7.4) son: 0 Éxito, 1 Versión, 2 Solicitud, 3 No hallado, 4 No servible, 5 No permitido, 6 Capacidad y 7 Interno.

3.3. Conversión de Endianness y Manejo del Flujo TCP

Debido a que el protocolo TCP funciona como un flujo no delimitado de bytes y los procesadores pueden usar ordenamiento *Little-Endian*, se implementaron funciones auxiliares con semántica de lectura/escritura exacta y conversión explícita a *Big-Endian* mediante `htons()`,

Tipo	Mensaje	Sentido	Carga útil
1	SALUDO	Cliente → Servidor	Vacía
2	BIENVENIDA	Servidor → Cliente	Texto con los servicios (LISTAR,ABRIR,UNIR,SALIR,CERRAR); el cliente la descarta
3	LISTAR	Cliente → Servidor	Vacía
4	CATALOGO	Servidor → Cliente	Una línea por contenido: nombre\ttamaño\t1\n
5	ABRIR	Cliente → Servidor	Nombre del contenido, sin terminador nulo
6	ACEPTADO	Servidor → Cliente	8 bytes: tamaño total del archivo (entero de 64 bits)
7	DATOS	Servidor → Cliente	Bloque de contenido (hasta 64 KB)
8	FIN (MSG_FINAL)	Servidor → Cliente	Vacía
9	ERROR	Servidor → Cliente	Texto legible con la causa; el código va en el campo estado
10	CERRAR	Cliente → Servidor	Vacía
11	DESPEDIDA	Servidor → Cliente	Vacía, o Servidor detenido si el servidor se apaga
12	UNIR	Cliente → Servidor	Vacía (el cliente no puede elegir posición)
13	EMISION	Servidor → Cliente	16 bytes: posición dentro del ciclo en ms + duración del ciclo en ms (dos enteros de 64 bits)
14	SALIR	Cliente → Servidor	Vacía (el cliente actual no lo usa)

Cuadro 2: Mensajes del protocolo (Sección 7.3 del proyecto).

`htonl()`, `ntohs()` y `ntohl()`.

```c
static ssize_t leer_exacto(int fd, void *buf, size_t n) {
    size_t hecho = 0;
    while (hecho < n) {
        ssize_t k = read(fd, (char *)buf + hecho, n - hecho);
        if (k < 0) {
            if (errno == EINTR) continue; // Manejo de interrupciones por senal
            return -1;
        }
        if (k == 0) return 0; // EOF de red
        hecho += (size_t)k;
    }
    return (ssize_t)n;
}
```

Las demás funciones auxiliares del cliente son:

- `escribir_exacto(fd, buf, n)`: repite `write` hasta enviar los `n` bytes; reintenta ante `EINTR` y devuelve -1 ante error. También se usa para escribir en la tubería de VLC.
- `descartar(fd, n)`: lee y desecha `n` bytes en trozos de 256. Mantiene el flujo alineado cuando llega un mensaje cuya carga no se usa, como `BIENVENIDA` o `DESPEDIDA`.
- `be64(b)`: convierte 8 bytes en orden de red a un entero de 64 bits. Es necesaria porque `htonl`/`ntohl` solo cubren 32 bits; se usa para el tamaño de `ACEPTADO` y para la posición y el ciclo de `EMISION`.

4. Implementación de las Acciones Principales

4.1. 1. Inicio de Sesión y Saludo Inicial

Todo flujo operativo del cliente comienza con el apretón de manos inicial: envía un mensaje `MSG_SALUDO` y valida que la primera respuesta del servidor sea de tipo `MSG_BIENVENIDA`, cuya

carga útil se descarta con `descartar()`. Si el servidor responde con `MSG_ERROR` (por ejemplo, *Servidor lleno: límite de clientes alcanzado* o una versión incompatible), el cliente muestra el texto del error por la salida de error, cierra el socket y termina con código `2`.

4.2. 2. Acción de Consulta de Catálogo (`listar`)

Envía el mensaje `MSG_LISTAR` (tipo 3, longitud 0). Al recibir la respuesta de tipo `MSG_CATALOGO`, el cliente reserva memoria dinámica exacta (`malloc(len + 1)`), lee la carga con `leer_exacto()`, la finaliza con el carácter nulo (`\0`) y la imprime en consola: una línea por contenido con nombre, tamaño en bytes y disponibilidad. Después libera la memoria con `free()`. Si la respuesta es `MSG_ERROR`, termina con código `3`.

4.3. 3. Acción de Descarga Bajo Demanda (`descargar`)

1. **Solicitud:** Envía la cabecera `MSG_ABRIR` adjuntando el nombre del archivo en la carga útil.
2. **Rechazo:** Si la respuesta es `MSG_ERROR` (contenido inexistente, nombre inválido o no servible), muestra la causa y termina con código `3` sin crear el archivo de destino.
3. **Confirmación:** Si la respuesta es `MSG_ACEPTADO`, verifica que la carga mida 8 bytes y obtiene con `be64()` el tamaño exacto del archivo. Recién entonces crea el archivo de destino con `fopen`.
4. **Transferencia por Bloques:** Lee en un bucle mensajes `MSG_DATOS`, escribe el *payload* en disco mediante `fwrite()` y acumula los bytes en `tam_recibido`.
5. **Cierre e Integridad:** Al recibir `MSG_FINAL`, compara `tam_recibido` con el tamaño anunciado (RF-37). Si la transferencia se interrumpe (conexión caída, `MSG_ERROR`, `MSG_DESPEDIDA` porque el servidor se detuvo) o los tamaños no coinciden, invalida la descarga con `remove(destino)` y termina con código `2` (RF-36).
6. **Visualización:** Si la bandera `-n` no está activa y VLC está instalado, invoca `lanzar_vlc()` con la ruta del archivo descargado. Si VLC no está instalado, lo informa y deja el archivo guardado.

4.4. 4. Acción de Canal en Vivo (`canal`)

Envía el mensaje `MSG_UNIR` (tipo 12). Si el servidor responde con `MSG_EMISION`, verifica que la carga mida 16 bytes, lee con `be64()` la posición de incorporación y la duración del ciclo, y las muestra en consola. Luego:

- Si la bandera `-n` no está activa y VLC está instalado, ignora `SIGPIPE` y llama a `lanzar_vlc()` para obtener el extremo de escritura de una tubería conectada a la entrada de VLC.
- Entra en un bucle de recepción de bloques `MSG_DATOS`. Los bytes leídos de la red se escriben con `escribir_exacto()` en la tubería, de modo que VLC los reproduce mientras llegan. Sin VLC, los bytes se reciben y se descartan.
- El bucle termina cuando el usuario cierra VLC (la escritura en la tubería falla y el cliente informa *Se cerró VLC*), cuando el servidor envía `MSG_DESPEDIDA` (*El servidor se detuvo*) o `MSG_ERROR`, o cuando se pierde la conexión. También puede interrumpirse con Ctrl+C.
- Al salir del bucle cierra la tubería; VLC recibe el fin del flujo y se cierra al terminar lo que tiene en memoria, gracias a la opción `-play-and-exit`.

Como el cliente se incorpora a la mitad de un archivo, los videos del canal deben estar en formato MPEG-TS (`.ts`), que admite empezar la reproducción en cualquier punto. Un MP4 no puede decodificarse empezando a la mitad.

4.5. 5. Lanzamiento de VLC (`lanzar_vlc`)

`vlc_disponible()` recorre las rutas de la variable `PATH` buscando un ejecutable `vlc`. `lanzar_vlc(archivo, fd_entrada, sockfd)` crea el proceso del reproductor:

- **Canal** (`archivo == NULL`): crea una tubería con `pipe`, hace `fork`, y en el hijo conecta el extremo de lectura a la entrada estándar con `dup2` antes de ejecutar `vlc -play-and-exit -`. El padre conserva el extremo de escritura.
- **Descarga** (`archivo != NULL`): el hijo llama a `setsid()` para que VLC siga abierto aunque el cliente termine, y ejecuta `vlc -play-and-exit <archivo>`.
- En ambos casos el hijo cierra el socket del servidor (la conexión pertenece al cliente) y redirige la salida de VLC a `/dev/null` para que no se mezcle con los mensajes del cliente. Si `execlp` falla, el hijo termina con `_exit(127)`.

4.6. 6. Cierre de la Sesión

Al terminar cualquiera de las acciones, el cliente envía `MSG_CERRAR`, cierra el socket y termina con código `0`.

5. Helpers

Este módulo ofrece dos grupos de funciones auxiliares de la capa de servicio (`server/catalogo.c` y `server/sesiones.c`), que usa el servidor: las del catálogo, que describen *qué contenidos existen*, y las de sesiones, que describen *una transferencia en curso para un cliente*. Las que pueden fallar devuelven un código de estado (`estado_t`) y entregan sus resultados mediante parámetros de salida. Cuando el código no es `SRV_OK`, los parámetros de salida no son válidos.

5.1. Funciones del catálogo

El catálogo es una estructura opaca que contiene un arreglo de entradas internas (nombre, tamaño y ruta física) y, para el canal en vivo, un arreglo de duraciones en milisegundos. Se construye una sola vez al iniciar el servidor y después es de solo lectura, por lo que varios hilos o procesos pueden consultarlo sin exclusión mutua.

`catalogo_escanear(ruta, out)`

Recorre el directorio indicado y construye el catálogo.

- Rechaza argumentos nulos o una ruta vacía (`SRV_ERR_ARGUMENTO`) y devuelve `SRV_ERR_IO` si el directorio no se puede abrir.
- Ignora los nombres que empiezan con punto (incluidos `.` y `..`), los nombres que no caben en `LARGO_NOMBRE`, los archivos cuya ruta completa (`directorio/nombre`) no cabe en `LARGO_NOMBRE` (50 bytes), y todo lo que no sea un archivo regular legible de tamaño mayor que cero. Para esto usa `lstat`, de modo que los enlaces simbólicos no se siguen.
- Guarda para cada entrada válida el nombre lógico, el tamaño y la ruta física; esta última nunca sale del módulo.
- Ordena las entradas alfabéticamente, para que el orden del canal sea el mismo en cada ejecución.
- Si no queda ninguna entrada devuelve `SRV_ERR_CATALOGO_VACIO`.
- Solo asigna `*out` cuando todo salió bien; ante cualquier error libera lo reservado y deja la salida intacta.

`catalogo_cargar_duraciones(cat)`

Lee el archivo de duraciones del canal, con un entero por línea, en milisegundos y en el mismo orden alfabético del catálogo. La ruta es fija: `../videos/.duraciones.txt`, relativa al directorio de trabajo, por lo que el servidor debe ejecutarse desde un directorio hermano de `videos/`.

- Ignora las líneas en blanco.
- Devuelve `SRV_ERR_IO` si una línea contiene caracteres que no son un número, si el valor desborda o si, al final, la cantidad de duraciones no coincide con la cantidad de entradas.
- Es todo o nada: con cualquier línea inválida no se guarda nada y el catálogo queda como estaba.
- No valida que cada duración sea mayor que cero; eso lo hace después `canal_calcular_punto`.
- No comprueba si `fopen` devuelve `NULL`, ni limita la escritura en el arreglo al número de entradas: un archivo inexistente, o uno con más líneas que entradas, produce un acceso inválido a memoria antes de que se compare la cantidad. El `main` del servidor comprueba que el archivo exista antes de llamarla.
- Debe llamarse una vez, tras `catalogo_escanear` y antes de crear hilos o procesos.

`catalogo_cantidad(cat)`

Devuelve el número de entradas del catálogo, o 0 si `cat` es `NULL`.

`catalogo_obtener(cat, indice)`

Devuelve un puntero constante a la entrada `indice` (`const EntradaInterna *`), con nombre, tamaño y ruta física.

- Devuelve `NULL` si `cat` es nulo o si el índice está fuera de rango.
- El puntero apunta al arreglo interno: es válido mientras no se libere el catálogo. Quien lo usa no debe exponer la ruta al cliente; el servidor solo envía nombre y tamaño.

`entrada_nombre(en, nombre)`

Declarada en `catalogo.h` pero sin implementación; no se usa.

`catalogo_duraciones(cat)`

Función de uso interno. Devuelve el arreglo de duraciones en milisegundos, o `NULL` si no se han cargado. No admite `cat` nulo.

`catalogo_liberar(cat)`

Libera primero el arreglo de duraciones, luego el de entradas y por último la estructura. Acepta `NULL`. No debe llamarse mientras existan sesiones abiertas, porque las sesiones de canal conservan un puntero al catálogo.

5.2. Funciones de sesiones

Una sesión es el estado de una transferencia en curso para un solo cliente: modo (bajo demanda o canal), descriptor de archivo, tamaño del archivo actual, posición dentro de él y, en el canal, el catálogo y el índice del archivo actual del ciclo. Cada conexión crea la suya, por lo que no hay estado compartido entre clientes. El contenido se trata siempre como bytes opacos: no se interpreta el formato del video.

`abrir_entrada(entrada, fd, tam)`

Auxiliar interno compartido por las demás funciones.

- Abre la ruta física de la entrada en solo lectura con `O_CLOEXEC`.
- Traduce los errores de apertura: `ENOENT` a `SRV_ERR_NO_ENCONTRADO`, `EACCES` a `SRV_ERR_NO_SERVIBLE` y cualquier otro a `SRV_ERR_IO`.
- Verifica con `fstat` sobre el descriptor ya abierto que sea un archivo regular de tamaño mayor que cero, lo que evita la ventana entre comprobar y abrir.
- Entrega el descriptor y el tamaño real; si falla, no deja ningún descriptor abierto.

`sesion_abrir_demanda(cat, nombre, out, tamano_total)`

Abre un contenido del catálogo para transferirlo completo desde el byte 0.

- Rechaza argumentos nulos (`SRV_ERR_ARGUMENTO`) y, antes de validar el nombre, deja `*out` en `NULL` y `*tamano_total` en 0.
- Valida el nombre, que proviene de la red: rechaza el nombre vacío, los que no caben en `LARGO_NOMBRE` y los que contienen `/` o `..` (`SRV_ERR_NOMBRE_INVALIDO`).
- Busca el nombre en el catálogo (`SRV_ERR_NO_ENCONTRADO` si no existe) y abre el archivo con la ruta guardada, nunca con una ruta construida a partir del nombre del cliente.
- Reserva la sesión con posición 0 y entrega el tamaño total para que el cliente sepa cuánto esperar. Si falla la reserva, cierra el descriptor antes de devolver el error.

`sesion_abrir_canal(cat, transcurrido_ms, out)`

Abre el canal en vivo en el punto que corresponde al tiempo transcurrido desde el inicio del canal.

- Exige un catálogo no vacío con las duraciones ya cargadas y un tiempo no negativo.
- Calcula con `canal_calcular_punto` qué archivo del ciclo está al aire y cuántos milisegundos van dentro de él.
- Convierte ese desfase en bytes con la proporción tamaño·offset_ms/duración_ms, usando aritmética de 128 bits para evitar desbordamiento, y posiciona el descriptor con `lseek`.
- Guarda en la sesión el catálogo y el índice del archivo actual para poder avanzar después.
- Con cualquier error cierra el descriptor y deja `*out` en `NULL`.

`sesion_leer_bloque(s, buf, max, leidos)`

Entrega el siguiente bloque de datos de la sesión.

- Devuelve `SRV_ERR_ARGUMENTO` si `s` o `buf` son nulos o si `max` es 0.
- Nunca lee más allá del final del archivo actual; el bloque puede ser más corto que `max`, y quien lo usa debe tomar siempre `*leidos` como longitud real.
- En el modo bajo demanda, al llegar al final devuelve `SRV_OK` con `*leidos == 0`.
- En el modo canal no hay final: al terminar un archivo abre el siguiente del ciclo (volviendo al primero tras el último) y continúa desde su byte 0. Si la apertura falla, la sesión sigue siendo válida.
- No controla el ritmo: entrega los bytes tan rápido como se le pidan. El ritmo de reproducción del canal lo impone el servidor (`server/atencion.c`), que solo pide los bytes que corresponden al tiempo transcurrido.
- Reintenta la lectura si una señal la interrumpe (`EINTR`) y trata una lectura de 0 bytes antes del final esperado como `SRV_ERR_IO`, porque indica que el archivo fue truncado.

`sesion_cerrar(s)`
: Cierra el descriptor y libera la sesión. Acepta `NULL`. Debe llamarse al terminar la transferencia y también si el cliente se desconecta a mitad.

`canal_calcular_punto(duraciones, n, transcurrido_ms, out)`
: Función pura del modelo temporal: no lee el reloj, recibe el tiempo como parámetro, lo que permite probarla con valores controlados.

  - Suma las duraciones del ciclo vigilando el desbordamiento, reduce el tiempo con aritmética modular y recorre las duraciones para ubicar el archivo y el desfase dentro de él.
  - Un instante exactamente en la frontera entre dos archivos pertenece al inicio del siguiente.
  - Devuelve `SRV_ERR_ARGUMENTO` si no hay duraciones, el tiempo es negativo, alguna duración es menor o igual que cero o la suma desborda.

`reloj_monotonico_ms()`
: Devuelve los milisegundos del reloj `CLOCK_MONOTONIC`, que no retrocede aunque se cambie la hora del sistema. Solo las diferencias tienen sentido; devuelve -1 si el reloj falla.

6. Compilación y Uso

El cliente depende solo de la biblioteca C estándar; para reproducir video requiere además `VLC` instalado en el equipo cliente. Se compila junto con el servidor desde la raíz del proyecto, o por separado:

```
make                        # produce bin/servidor y bin/cliente
gcc -std=c11 -D_DEFAULT_SOURCE -O2 -static client/cliente.c -o cliente
```

La versión estática (`-static`) no depende de las bibliotecas del sistema y puede copiarse como un solo archivo a otro equipo Linux de 64 bits.

Opción	Valor	Uso
`-h`	IPv4 numérica	Dirección del servidor (obligatoria)
`-p`	1–65535	Puerto del servidor (obligatoria)
`-a`	`listar` | `descargar` | `canal`	Acción (obligatoria)
`-f`	Nombre de contenido	Contenido a descargar, tal como aparece en `listar` (solo `descargar`)
`-d`	Ruta de archivo	Dónde guardar la descarga (solo `descargar`)
`-n`	—	No abrir VLC
`-help`	—	Muestra la ayuda y termina

Cuadro 3: Parámetros del cliente.

```
./cliente -h 127.0.0.1 -p 8080 -a listar
./cliente -h 127.0.0.1 -p 8080 -a descargar -f pelicula1.mp4 -d ./pelicula1.mp4
./cliente -h 127.0.0.1 -p 8080 -a canal
./cliente -h 127.0.0.1 -p 8080 -a canal -n
```

7. Códigos de Salida

8. Verificación

El cliente se verificó en los tres modos del servidor con las pruebas automáticas del proyecto, que están en la carpeta `tests`: la batería (`bateria.py`) y las pruebas de argumentos

Código	Constante	Cuándo
0	`EXIT_SUCCESS_CUSTOM`	La acción terminó correctamente, o se mostró la ayuda
1	`EXIT_ARGS_ERROR`	Argumentos ausentes o inválidos, acción desconocida, o destino de descarga que no se puede crear
2	`EXIT_NET_ERROR`	No se pudo crear el socket o conectar, IP inválida, el servidor rechazó la conexión (incluido el rechazo por capacidad), respuesta malformada, o descarga interrumpida o incompleta
3	`EXIT_CONTENT_ERROR`	El servidor rechazó el contenido pedido en `listar` o `descargar` (no hallado, nombre inválido, no servible)

Cuadro 4: Códigos de terminación del cliente (RF-29).

(`argumentos.py`).

- **T28**: `listar` termina con código 0 y muestra todas las entradas, sin rutas internas.
- **T29**: `descargar` de un archivo de 30 MB termina con código 0 y el archivo guardado es idéntico al original (SHA-256).
- **T30**: `descargar` de un contenido fuera del catálogo termina con código 3 y no deja archivo de destino.
- **T31**: `canal` muestra la posición de incorporación y, al interrumpir el cliente con Ctrl+C, el servidor sigue atendiendo.
- **Argumentos**: 7 invocaciones (sin argumentos, acción desconocida, `descargar` sin `-f`/`-d`, puerto fuera de rango, IP inválida, servidor inexistente y ayuda) terminan con el código esperado.
- Compila sin avisos con `gcc` y `clang` usando `-Wall -Wextra -Werror`, y sin errores de memoria bajo AddressSanitizer.
- La integración con VLC se verificó con VLC 3.0.20: reconoce el flujo MPEG-TS del canal, decodifica video H.264 y audio AAC, y se cierra solo al terminar el flujo.

9. Limitaciones Conocidas

- **Interrupción con Ctrl+C durante una descarga** (RF-36, RF-70): no hay manejador de `SIGINT`, por lo que el archivo parcial queda en disco con su nombre final.
- **Códigos de salida** (RF-68): no hay códigos propios para el rechazo por capacidad ni para el error interno; ambos terminan con código 2.
- **Destino no escribible** (RF-64, RF-67): se detecta al recibir `ACEPTADO`, después de conectarse, y no antes de iniciar la operación de red.
- **Dirección del servidor**: solo se acepta una IPv4 numérica (`inet_pton`); la Sección 11.2 admite también un nombre resoluble.
- **Puerto**: se lee con `atoi`, que acepta entradas como `8o80` (se interpreta como 8) sin informar el error (RF-65).
- **Ayuda** (RF-66): no indica valores por omisión, y ante argumentos inválidos se imprime por la salida estándar.
- **Parámetros opcionales de la Sección 11.2**: no se implementan el tiempo límite ni el tiempo de canal.

- **Canal**: el cliente nunca envía `SALIR`; abandona el canal cerrando la conexión. Si el servidor rechaza la incorporación con `ERROR`, el cliente termina con código 0.
- **Memoria**: el `malloc` de `listar` no comprueba si devuelve `NULL`.